Name: P H Devesh

Register Number: 1911260236

Subject: Python Programming (CSA0805)

Assignment: CO3

Q1 – Lists and Slicing: Daily Rainfall Records

Data design

A list is appropriate for 365 ordered daily readings because indexing and slicing are direct. readings[7:14] returns days 8–14, readings[-7:] returns the final seven values, and readings[::7] samples every seventh item. max/min with enumerate preserve the index; dry days are indices whose value is zero.

Moving average pseudocode

IF length(readings) < 7 THEN return empty list. SET window_sum to sum of first seven readings. APPEND window_sum/7. FOR each next index: subtract the value leaving the window, add the new value, append the average. RETURN results.

Complexity

The naive method sums seven values for each output: O(7n)=O(n), but performs about 7(n–6) additions. The running-sum method uses an initial 7 additions and two updates per later day, O(n) with fewer operations and O(1) extra working space beyond output. list.append is amortised O(1), insert/pop at the front are O(n), and sort is O(n log n).

Late readings and alternatives

list.insert at an interior index shifts later elements and costs O(n). For a streaming seven-day window, deque(maxlen=7) is suitable. A date-keyed dictionary is useful for late corrections and direct date lookup; it does not by itself preserve chronological order for calculations.

Python Program

```
import random, time

def moving_naive(values, window=7):
    return [sum(values[i:i+window]) / window
            for i in range(len(values)-window+1)]

def moving_running(values, window=7):
    if len(values) < window:
        return []
    current = sum(values[:window])
    result = [current / window]
    for i in range(window, len(values)):
        current += values[i] - values[i-window]
        result.append(current / window)
    return result

if __name__ == "__main__":
    random.seed(10)
    for n in (365, 36_500, 365_000):
        x = [random.uniform(0, 50) for _ in range(n)]
        for fn in (moving_naive, moving_running):
            start=time.perf_counter(); out=fn(x); elapsed=time.perf_counter()-start
            print(n, fn.__name__, len(out), f"{elapsed:.6f}s")
```